From sequences to transformers

NLP, part 2. How models learned to read in context, and when a small one beats a big one.

60 min | Recorded | Class 5 of 15

5

What this session covers

01
Reading in order
2 slides

02
Attention
3 slides

03
Two families, and how they're trained
3 slides

04
Small model or big model
3 slides

05
Code walkthrough
5 slides

© 2026 Devansh Parapalli

Section 1 of 5

Reading in order

The first attempts at context.

01

Recurrent networks read one word at a time and carry a memory forward

02

LSTMs added gates to decide what to keep, which helped but didn't fix speed

© 2026 Devansh Parapalli

Recurrent networks read one word at a time and carry a memory forward

An RNN reads "receipt failed after reopen" word by word, updating a small memory after each one. By the end, that memory holds a summary of the sentence.

Word order finally counts.

The problem: the memory fades. By word 200 of a payroll thread, word 3 is mostly gone. And reading one word at a time is slow to train.

© 2026 Devansh Parapalli

LSTMs added gates to decide what to keep, which helped but didn’t fix speed

Gates decide what to write into memory, what to keep and what to forget. Long tickets hold together much better.

For a few years, this was how translation and speech worked.

Still: one word after another. Training can’t spread across thousands of GPUs, so models stayed small.

© 2026 Devansh Parapalli

Section 2 of 5

Attention

The idea the whole of generative AI rests on.

01

Attention lets “it” find “close job” in the same sentence

02

A transformer stacks attention layers, and it trains in parallel

© 2026 Devansh Parapalli

Every word looks at **every other word**, all at once.

Instead of reading in a line, each word looks at every other word and decides which ones matter to it.

© 2026 Devansh Parapalli

Attention lets “it” find “close job” in the same sentence

The **close job** failed after the journal import because **it** timed out.

To understand “it”, the model weighs every other word. “Close job” gets most of the weight, “journal import” some, “the” almost none.

Each layer does this for every word at once, many times over.

© 2026 Devansh Parapalli

A transformer stacks attention layers, and it trains in parallel

Attention, then a small network per word, repeated dozens of times. Because every word is handled at once, training spreads across huge numbers of GPUs.

That's what made very large models possible.

The cost: every word compares itself with every other word, so work grows with the square of the length. That's why models have a context window (class 6).

© 2026 Devansh Parapalli

Section 3 of 5

Two families, and how they're trained

Encoders, decoders, pretraining and fine-tuning.

01

Encoders understand text; decoders write it

02

Pretrain once on everything, then fine-tune on your task

03

A sentence embedding turns a whole ticket into one vector that keeps its meaning

© 2026 Devansh Parapalli

Encoders understand text; decoders write it

Encoder, BERT-style

- Reads the whole text in both directions
- Produces a vector for the text
- Good at classifying, searching, matching

Decoder, GPT-style

- Reads left to right, predicts the next token
- Produces more text
- Good at writing, summarising, following instructions

© 2026 Devansh Parapalli

Pretrain once on everything, then fine-tune on your task

The expensive part is done by someone else.

01 Pretraining
Predict hidden or next words across a huge amount of public text. Weeks, many GPUs.

02 Fine-tuning
Train a little more on a few thousand labelled tickets. Hours, one GPU or a CPU.

03 Or neither
Use a pretrained model as it is: embeddings for search, a prompt for the rest.

© 2026 Devansh Parapalli

A sentence embedding turns a whole ticket into one vector that keeps its meaning

An encoder reads the ticket in context and returns one vector. “Import stuck” and “invoices not showing” land close together, and “down” means different things in different sentences.

This fixes the one-meaning-per-word problem from class 4. Class 9 builds retrieval on it.

numbers per ticket 1,024

per ticket on a laptop about 20 ms

© 2026 Devansh Parapalli

Section 4 of 5

Small model or big model

The same tickets, classified three ways.

01

On a fixed task with labels, the small models match or beat the LLM for far less

02

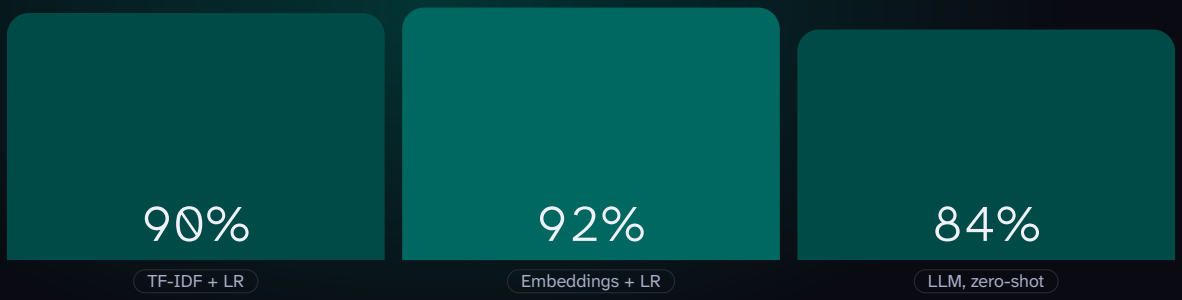
The gap in accuracy is small next to the gap in cost and speed

03

Pick the small model when the task is fixed, and the LLM when it keeps changing

© 2026 Devansh Parapalli

On a fixed task with labels, the small models match or beat the LLM for far less



Source: TF-IDF measured on the synthetic set; embedding and LLM figures illustrative

© 2026 Devansh Parapalli

The LLM loses on the vague tickets: it has never seen how LOL's analysts label "numbers look wrong in PayGrid". The small models learned it from history.

The gap in accuracy is small next to the gap in cost and speed

METHOD	ACCURACY	PER TICKET	PER 100K TICKETS
TF-IDF + LR	90%	under 1 ms	effectively free
Embeddings + LR	92%	about 20 ms	a few CPU hours
LLM, zero-shot	84%	about 800 ms	pay per token

Source: TF-IDF measured; other rows illustrative

© 2026 Devansh Parapalli

Pick the small model when the task is fixed, and the LLM when it keeps changing

Small task model

- One job, done the same way every time
- Labelled examples exist
- High volume, low cost, data stays local

General LLM

- The task changes or needs writing
- No labels, or not enough
- Volume is modest and someone checks the output

© 2026 Devansh Parapalli

Section 5 of 5

Code walkthrough

Embeddings with a simple classifier, then the LLM on the same tickets.

01
Local embeddings plus logistic regression, and no data leaves the machine

02
The LLM version has no training step, and a cost on every call

03
Run all three on the same test set before choosing, and write down why

04
Where today sits on the card

© 2026 Devansh Parapalli

Local embeddings plus logistic regression, and no data leaves the machine

code/class05/embed_classify.py

```
enc = SentenceTransformer("Qwen/Qwen3-Embedding-0.6B")
X_train, X_test = enc.encode(list(train.text)), enc.encode(list(test.text))
clf = LogisticRegression(max_iter=1000).fit(X_train, train.queue)
print(f"accuracy {clf.score(X_test, test.queue):.3f}")
```


© 2026 Devansh Parapalli

The LLM version has no training step, and a cost on every call

code/class05/llm_classify.py

```
def classify(text: str) -> str:
    prompt = f"Route this LOL service desk ticket to exactly one of {QUEUES}. Reply with the queue name only.\n\n{text}"
    return ask(prompt, temperature=0).strip().lower()

preds = [classify(t) for t in sample.text]
```


© 2026 Devansh Parapalli

Run all three on the same test set before choosing, and write down why

- Accuracy per queue, not just overall
- Time per ticket, measured, not guessed
- Cost per 100,000 tickets at today's prices
- Where the ticket text goes for each option (class 8)

© 2026 Devansh Parapalli

Where today sits on the card

1. A script	The steps are known and don't change.
2. A rule	The decision is known and must be explainable line by line.
3. Classical ML	Labelled history, and the input is a table or short text.
4. A small trained model	**Lots of text, images or audio, enough labels, cost per call matters.**
5. An LLM with a prompt	**Varied language in and out, few labels, someone checks the output.**
6. An LLM plus retrieval, tools or agents	Needs private knowledge, has to act, or takes several steps.
7. Not yet	No ground truth, needs a guarantee, or nobody owns the result.

© 2026 Devansh Parapalli

What to do next

1. Run both scripts in `code/class05` and put your numbers in the comparison table.
2. Note which queue each method gets wrong most.
3. Read the class 6 pre-reading on tokens.

Questions after class: the course channel.

© 2026 Devansh Parapalli